

# NBN Workshop Review Report

## Sprint 1 deliverable D2 | Software Development Lifecycle Using AI

Team 2, Group 20 · COSC2408 Programming Project 1 · 13 August 2026

### 1. Summary

The June workshop was run four times across four days for two audiences. It was pitched technically on Monday and Tuesday for a room of non-technical people, then re-pitched for non-technical people on Wednesday and Thursday for a room of engineers. Alessio said so himself in the closing remarks: "we hit the opposite." Both groups got material aimed at somebody else.

That is the single most useful thing in the recordings, and it is the reason this capstone exists. It also reframes the question Team 2 has been treating as a coin toss. The workshop did not fail because it chose the wrong audience. It failed because it had one layer and two audiences.

The content itself holds up. Alessio delivered a red, yellow and green classification for every stage of the lifecycle, walked eight stages end to end against a worked example, and made a specific claim about how the boundaries move over time. That framework should be the spine of our model rather than something we invent in Sprint 2.

What the workshop did not deliver is equally clear. Deployment was never reached. Legacy code, which Alessio named as the first thing a large organisation would use AI for, was described but never shown. Accountability for machine-written code was never raised. And collaboration, the thing most teams in both cohorts named as its biggest obstacle, was never taught at all. What exists came from the teams themselves, who invented four workable patterns between them while nobody was looking.

This draft adds the Tuesday recording, which was unusable when the report was first written and has since been regenerated in English. It matters more than its length suggests. Tuesday is the Group 1 showcase, the twin of the Thursday session the report was already built on, so every finding in section 7 can now be tested against two rooms instead of one. Most of them replicate, which turns them from anecdotes into interesting findings that recur across both cohorts. Tuesday also supplies the two things the first draft said were missing: a serious answer on technical guardrails, in section 4.5, and evidence that collaboration has candidate solutions, in section 7.3.

### 2. What this report is drawn from

This report is drawn from four recordings totalling 11 hours 19 minutes, transcribed through Microsoft Stream and cleaned. Roughly 3 hours 40 minutes of speech is captured, and the long silences have now been verified against the recordings by listening: they are real. The rooms were quiet because teams were building in breakouts or the session had ended while the recording ran on. Nothing said in these four recordings is missing from the transcripts.

| Recording        | Length | Speech captured | Note                                                                                                  |
|------------------|--------|-----------------|-------------------------------------------------------------------------------------------------------|
| Monday afternoon | 3h 02m | 1h 01m          | The demonstration is captured start to finish. The recording then runs through breakout time and ends |

| Recording          | Length | Speech captured | Note                                                                                                                                                                                      |
|--------------------|--------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |        |                 | before the room reconvenes; the silent tail was checked by listening and holds no speech                                                                                                  |
| Tuesday session 2  | 2h 05m | 1h 24m          | Originally transcribed through a French model and unusable. Regenerated in English and now the largest speech capture in the set. One 28-minute silence early, then continuous to the end |
| Thursday session 1 | 4h 00m | 15m             | Not a gap. A briefing at the start, then a mostly silent main room while teams built in breakouts. Verified against the recording                                                         |
| Thursday session 2 | 2h 11m | 1h 00m          | The session ends at 1h 17m with thanks and goodbyes. The remaining 54 minutes of recording are silent, checked by listening                                                               |

Two things follow. Both cohort showcases are captured in full, Group 1 on Tuesday and Group 2 on Thursday, alongside the Monday demonstration. Between them they carry the framework, the pain points, and two independent runs at the same exercise, which is most of what this report needs. What is missing is not in these recordings at all: session 1 for Monday and Tuesday, and everything for Wednesday. Whether those sessions were recorded is an open question for Alessio.

One note on reading the quotations. Stream heard "Claude" as "cloud" throughout. Corrections were applied only where the phrasing made it certain.

### 3. How the workshop was run

The workshop ran as two cohorts. Monday and Tuesday were Group 1, Wednesday and Thursday were Group 2. Each cohort got a demonstration day followed by a hands-on day where teams of five or six built a working application against a brief, then presented it back.

The exercise was a customer fault reporting tool: check service availability at an address, show available plans and providers, and recommend modem placement from a floor plan. A change request was introduced part way through the build to test how teams absorbed scope changes. Teams were assigned roles matching a delivery pod, with a project manager, business analyst, UX designer, developer and tester.

Around 32 to 35 people attended each day. Facilitation was shared between Alessio Bonti from RMIT, who ran the technical content, and Vivek from NBN, who ran the room and the playbacks. Leon appears as the design contact.

Both build days closed with a showcase in which every team presented what it had made and what had gone wrong. Group 1 presented on Tuesday afternoon and Group 2 on Thursday, six teams each. These two sessions are the closest thing the recordings hold to a controlled comparison: the same brief, the same tools, the same change request, two different rooms.

The workshop was also rebuilt between the cohorts, deliberately and mid-delivery. Alessio told the Tuesday room that the next two days would be different: "for tomorrow we are going to redesign the workshop a little bit based on your feedback, because we are finding that perhaps the actual talking

about the technology as a development technology is not as interesting or as fun as if we talk about everything else apart from the development." A co-facilitator on the same call put it more bluntly: "we are changing the entire picture." Section 8 returns to what that decision cost.

## 4. The framework

Alessio opened the Monday afternoon session by restating the classification the whole module runs on, and he was explicit that it applies to every stage:

| Classification | Meaning                                            |
|----------------|----------------------------------------------------|
| Red            | Human decision, judgement and approval             |
| Yellow         | AI assists. The AI drafts and the human approves   |
| Green          | AI automates. The AI executes and a human monitors |

*"Every stage we go through gets classified with these three."*

The part that matters most for our model came at the end of Thursday, and it is not in the framework as first presented. The boundaries move.

*"As the technology progresses, but also as you build the environment and the environment learns from you, you can slowly perhaps move some of those yellow tasks into the green area, because something that was assisted before, now you improve the governance, you also improve the process itself. Then slowly it starts moving into that green automation space, which is what we have done for the last 50 years."*

He tied that directly to the organisational rules file: "If we think about that CLAUDE.md file, which basically says these are all the rules for the organisation, as time progresses and that becomes richer and richer, we will have to do less things, because the system will know exactly what we are trying to do."

So the classification is not a static map. A stage sits at red because the governance and the context are not yet good enough to move it, and improving either one moves it. That gives our lifecycle map a second axis and it gives the governance module a job beyond compliance.

### 4.1 The eight stages

Eight stages were walked in sequence against the fault reporting tool. Seven were demonstrated live. The classifications below are as Alessio gave them, with his reasoning where he offered it.

| Stage                         | Classification                            | What was shown                                                                                                                                                                                                                                         |
|-------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Ideation and business case | Red, with a yellow first draft            | "Today this stage is almost entirely human." The AI drafted a business case with problem statement, proposed solution, three benefits with metrics, effort estimate and primary risk, in under a minute. Stakeholder alignment and sign-off stay human |
| 2. Requirements and discovery | Green for the first draft of user stories | The only green awarded in the whole walkthrough, and the reasoning is worth keeping: "everything that was probably supposed to be invented has already been invented. It is very rare that we find something that is                                   |

| Stage                               | Classification                                                | What was shown                                                                                                                                                                                                                                                                               |
|-------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     |                                                               | extremely novel. All of the knowledge in building these stories is coming from hundreds and thousands of companies that are very similar to NBN"                                                                                                                                             |
| 3. Solution design and architecture | Red as delivered, yellow if context improves                  | The output named a human decision point on architecture. Alessio disagreed with his own demo: "I would have loved to see this more on the yellow. If you have a good knowledge of the existing infrastructure, this will probably go down to yellow and that will save people a lot of time" |
| 4. UX design and prototyping        | Green for journey mapping, red for brand and design decisions | Emotional journey map, then a usability and accessibility critique pass. The critique framing was that you should not review your own work: "you are not the user"                                                                                                                           |
| 5. Development planning             | Yellow                                                        | MoSCoW prioritisation, story-to-role assignment across front end, back end and full stack, dependency flagging and complexity estimates, output as a two-week sprint plan and then pushed into GitHub issues                                                                                 |
| 6. Development and build            | Yellow                                                        | A GitHub issue drives the work. Claude Code reads the issue, confirms understanding of each acceptance criterion, plans, branches, implements, writes tests and opens a pull request                                                                                                         |
| 7. Testing and QA                   | Yellow                                                        | A review command runs an adversarial pass, surfaces issues for human decision, and iterates until tests pass. Alessio flagged the obvious hole: "the question would be, are all the tests correct?"                                                                                          |
| 8. Deployment and iteration         | Not reached                                                   | "We are not going to do the deployment because we do not have any deployment to deploy to"                                                                                                                                                                                                   |

## 4.2 Practices taught alongside

Several practices were taught alongside the walkthrough. These are the transferable parts and most of them belong in our model.

| Practice                                             | As presented                                                                                                                                                                                                              |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Write the issue properly                             | "If you write a good issue, you will get a good outcome." Acceptance criteria are the mechanism, because the agent iterates against them until they pass or it gives up                                                   |
| Always state what is out of scope                    | "If you do not tell it what you want, and also you do not tell it what you do not want, you will get into a huge scope creep." His example: "I wanted to make a small change and it ended up changing 35 different files" |
| Separate the instructing role from the building role | "I would really like to have a clear separation between people who are writing the code and people who are instructing on what to create and how to create it"                                                            |

| Practice                         | As presented                                                                                                                                                                                                                                                  |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Build a critique loop            | Have one group generate and another review, using deliberately harsh personas: "make sure to review this very deeply and ask the original creator to come back with a different option"                                                                       |
| Ask for options, not an answer   | Requesting three or five alternatives at design and UI stages was presented as faster than iterating on one                                                                                                                                                   |
| Context is the output            | "Anything created by AI depends on the quality of the data. Your context is what defines your output." The infrastructure map, API documentation and existing patterns are what move a stage from red to yellow                                               |
| Skills stop the reinvention      | On generating a deck: "the system does not know how to use PowerPoint. It can infer from its knowledge, but that means it has to reinvent all the time. The reason for calling the skill is that it teaches it how to read and write and design a PowerPoint" |
| Some commands escalate the model | Plan and review route to a stronger model automatically, and it is not a setting. To avoid it, phrase the request in plain language instead of using the command                                                                                              |

### 4.3 The brand colours incident

One exchange is worth recording in full because it is the clearest governance lesson in the workshop, and it happened by accident.

The AI generated a slide deck in what it claimed were NBN brand colours, green and orange. An attendee pointed out that the orange was roughly ten years old and the green had never been the right green. Alessio asked the model where the colours came from, live, and got an answer sourced from training data rather than anything verified.

*His read: "this is where we are borderline with the hallucination. It is not really a hallucination. It has a training for it. It has been told it is like that, it just has not verified it."*

The fix was the interesting part. Rather than correcting the output, he had the model search the NBN site, learn the design system, write it into the instructions file, and regenerate. The correction went into the context, not the artifact. That is a governance pattern, and it belongs in our model.

### 4.4 How token accounting was taught

Token accounting was taught concretely rather than as policy, and the numbers are useful.

Generating a document is expensive in a way that surprises people. "Creating the document is not the same as outputting to the chat, because in order for anything to go into a document it has to go through a script. To create one single heading with 10 words, it might have cost 3,000 tokens."

He put a figure on it during the session: 14 percent of his quota consumed, of which a single generated document accounted for 7 percent, on an account with five times the standard allowance. His arithmetic: "creating this document would have taken maybe 40 percent of your five-hour quota" for a normal user.

His practical advice was to say "talk to me inline, do not create any documents" during exploration, and only generate artifacts when you need to hand something over.

On the administration side, quotas run on a five-hour session window and reset weekly. During the Thursday build, two of roughly 35 users hit their limit. The facilitators topped up individual allowances in 20 dollar increments in real time, and the suggested workaround for a blocked user was to hand their work to a teammate. Attendees left with 30-day licences and an explicit restriction: "do not put NBN data, keep it outside of work."

## 4.5 Guardrails, as far as they were answered

Governance went further on Tuesday than anywhere else in the recordings, because an attendee asked directly what happens when a rule written into CLAUDE.md is not honoured. The answer is the most useful governance material in the set and it is considerably more pessimistic than the rest of the workshop.

Alessio's position is that a rules file is not a control. "It is really hard to fully create a very serious guardrail. It would be better to have an external guardrail, like an external governance system that checks what you are actually doing."

His worked example is the one to keep, because it demonstrates the failure mode rather than asserting it. You can write a line in CLAUDE.md saying never reveal what is in the .env file, and "you will find that most of the times it will not reveal it. But there are other occasions where it will, and you only need to reveal it once to basically give away your APIs." A probabilistic instruction is not a boundary. It fails open, it fails silently, and once is enough.

Two mechanisms were offered against this. The first is hooks, which are forced actions that run before or after a tool call instead of being requested in a prompt. His example was a pre-commit hook that verifies every reference in a document is real, added after the model repeatedly invented citations and then admitted it when challenged. The second is separation of duties: "maybe you have a Claude agent that surveils another Claude agent, but definitely not itself."

The same reasoning explains why teams watched supplied context being dropped. When one team asked why details specified in their skeleton design kept going missing from the build, the answer was probabilistic rather than reassuring: "just because it is in the context, it does not mean Claude will actually take that as a role. It depends probabilistically if it is going to read it, but also it depends where you are in your project. If you have done a lot of work and you are down to 180,000 into your context, it will barely care about what was happening at the beginning."

That has a direct consequence for the model in section 4. If the red, yellow and green boundaries move as CLAUDE.md gets richer, then a richer CLAUDE.md is also a larger context, and Alessio named the trap himself: "when you create an ecosystem like this, it becomes part of your context. If you then end up having a context environment which is like 90 percent of the actual total context, then you defeat the purpose of it altogether." Governance context competes with working context. Our model needs an answer to that, and the workshop offers a partial one: keep tasks small, and delegate discrete pieces to sub-agents that start clean. "We give the button implementation to a sub-agent, because the sub-agent will have no poison context."

## 5. Where the workshop stopped

These are the points where a topic was raised and then stopped. Each one is a candidate for what our model has to add.

| Topic                                      | How far it got                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deployment and iteration                   | Stage 8 of 8, never demonstrated. There was no environment to deploy to                                                                                                                                                                                                                                                                                                                                                                     |
| Legacy code                                | Named as the first and second things a large organisation would actually use AI for, ahead of writing new code, and then never shown. "You might have 50 years of legacy with different people, people that have not put comments in, people that have come up with their own ways of doing stuff"                                                                                                                                          |
| Enterprise-scale complexity                | Challenged directly from the floor and never resolved in practice. Covered in the next section                                                                                                                                                                                                                                                                                                                                              |
| Backend-only change                        | Raised in the closing discussion and explicitly parked. "How would you prototype a fully backend change?" The follow-up was that some systems should never go near this at all                                                                                                                                                                                                                                                              |
| Collaboration                              | The largest gap by distance. f in both cohorts named it. The facilitator's own verdict on Tuesday: "Claude still has a lot of room to grow as an enterprise system. It is not something that seems ready for proper group collaboration." No method was taught. What exists came from the teams themselves and is in section 7.3                                                                                                            |
| Verification of AI-written code            | Raised as a worry on Monday and half answered on Tuesday. "Are all the tests correct?" is still open, but one team's generated test suite did catch a real defect: the model had ignored the supplied mock data and populated the build with the real names of retail service providers. Verification worked. It was never taught                                                                                                           |
| Liability and authorship                   | Still not covered. Tuesday added a direct answer on technical guardrails, in section 4.5, and that answer is that the model cannot police itself. Who is accountable for machine-written code, and what authorship means for it, was never raised in either cohort                                                                                                                                                                          |
| Skills against commands against sub-agents | Deferred on Monday and skipped in the build: "we have not really used them because I think it was too complicated for today." Alessio's own view is that skills are "extremely underappreciated", and the one team that used them produced the clearest governance win of the two days, in section 7.3                                                                                                                                      |
| Requirements tooling integration           | A question about connecting Jira or Confluence got "there would probably be a connector" and moved on                                                                                                                                                                                                                                                                                                                                       |
| Multi-agent review                         | The idea of one agent generating and another critiquing was described as a possibility, never built                                                                                                                                                                                                                                                                                                                                         |
| Enterprise-scale change control            | Asked directly on Tuesday: how do you stop an agent editing files you did not ask it to, in a codebase of tens of thousands or millions of lines across microservices. The answer was hooks restricting which paths an agent may touch, and that demonstration had failed the previous day because the demo machine was Windows and the paths resolved differently. Named as a topic for a future partnership discussion rather than solved |

## 6. The challenge from the floor

The sharpest moment in the recordings came from a core application developer who was not persuaded, and his objection is the best specification we have for what a senior audience needs.

*"So far I have seen we were not even closer to what we do in day to day development, because what you are showing is a very simple API which any engineer can do in half an hour maximum. In our NBN it is a bigger enterprise application. Usually as an engineer, where we struggle in day to day work is where we spend a lot of time for the complex logic."*

He then made the point that greenfield work is not the job: "It is not like we will be creating a new API every day. We will be creating one API maybe for six months, and then after that we will keep enhancing the API. That will be more our day to day work."

And he gave the shape of the problem: payloads with nested attributes, collections of objects, conditional validation, and address or location search endpoints carrying 30 to 40 query parameters.

Alessio conceded the point and redirected rather than defending the demo. On the productivity figure: the 5 to 10 percent claim was deliberately conservative against vendor marketing of 80 percent. On sequencing, he argued that coding is not where a large organisation starts. Understanding and documenting the existing system comes first, refactoring second, and writing new code fourth or fifth. His caution on scale: "perhaps it is good if you go down the path of, can you please write this one API endpoint, rather than write an entire class that talks to another five different things."

A second data point landed on Thursday from a Team 1 member describing real NBN work rather than the exercise: "I recently undertook a project in anger for NBN which was a set of API endpoints. It did the boilerplate very, very well, but the code itself was absolute garbage, basically unusable and unreadable. I had to refactor and correct the whole thing."

A third voice came from the Tuesday room and pointed at change control rather than code quality. The questioner set the exercise aside as unrepresentative because it was greenfield, then described the real case: a codebase running to "millions or 10 thousands of lines" across microservices, and asked how you stop Claude Code or GitHub Copilot editing files you never asked it to touch. The exercise had been built to gesture at this, and Alessio said so: the change request existed precisely so that teams would have to modify something they had already built, and several teams reported the model obliging by editing places they had not anticipated. But the honest answer at real scale was hooks restricting which paths an agent may touch, and that demonstration had failed the previous day because the demo machine was Windows and the paths resolved differently.

Three independent senior voices across both cohorts, converging on the same place. The tools handle the boilerplate and fall over on the part that is actually hard, and the part that is actually hard is nearly always a change to something that already exists. Any model we produce that does not say this plainly will not be believed by the people who have to use it.

## 7. What the audience struggled with

Both cohorts closed their build day with every team presenting what they had made and reflecting on what had worked and what had been difficult, in front of the client. Group 1 presented on Tuesday and Group 2 on Thursday, six teams each. Twelve teams, two rooms, one brief. This is the most valuable evidence in the recordings, and where an obstacle appears in both cohorts, it is a less likely to be a one-off feature of a single session.



| Obstacle                  | Where it appeared                 | Evidence                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Collaboration             | Both cohorts, multiple teams      | Collaboration was repeatedly raised as a significant difficulty across both cohorts, "We wasted a lot of time, almost an hour trying to figure out how to get all that working." One Thursday team could not download files from the tool and copied content by hand into Teams. Tuesday went the same way: one team never got chat sharing working at all, reporting "we couldn't make it work in the 3 1/2 hours we spent." The clarification that would have helped arrived only near the end, from a facilitator: a project is "a shared workspace so that you and your project team members can understand the instructions and shared assets, but it is not intended as a place to merge code" |
| Tooling ambiguity         | Both cohorts, at least four teams | "A lack of clarity around the tooling, whether we needed VS Code or not, GitHub, and so on." One team installed GitHub and never used it. On Tuesday an engineer lost time blocked on pushing to a repository his team was never meant to push to, because the instruction to fork it had gone out by email and slide and had not landed. Alessio took it on the chin: "I put my hands up"                                                                                                                                                                                                                                                                                                           |
| Assumed knowledge         | Thursday, named explicitly        | "A lot of assumptions with knowledge of things like the terminal, using the terminal to put in your prompts. There is a lot of technical knowledge required just using the tools, and the terms within the tools, especially if you are coming in cold"                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Uneven skill inside teams | Thursday, Team 3                  | "I have not touched these tools pretty much at all. I have only ever really used Copilot at work, so never in this context." She worked alone on a side copy rather than trying to collaborate, because handing work back and forth was too hard                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Token limits mid-flow     | Both cohorts                      | "There is nothing that slowed us down quicker than one at a time us hitting our token limit. The momentum was great, and then suddenly we cannot access or do anything. I do not envy whose job it is to figure out how we adopt that as a strategy at NBN." On Tuesday a designer exhausted his allowance mid-session and handed the tool to a teammate to carry on                                                                                                                                                                                                                                                                                                                                 |
| Design fidelity           | Both cohorts                      | "Claude Code does not 100 percent reflect that UX design." Tuesday teams hit this harder because more of them worked from an explicit design: "even though we provided the skeleton, and the skeleton has correct information on the buttons and everything, somehow we did see some percentage Claude missed it, and we have to reiterate that specific information again." Closing the gap was manual in every case                                                                                                                                                                                                                                                                                |
| Machine setup             | Thursday, Team 1                  | "That person's machine was not optimised as a developer's machine and we paid the price for that." Their estimate was that a properly configured environment would have cut the build to under an hour                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| The 90 percent wall       | Tuesday, named twice              | Named twice in the session, in almost identical words: once during the team 1 discussion, "I generally found with the UI elements it would get to about 90 percent and then it would just                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| Obstacle                                | Where it appeared  | Evidence                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                         |                    | either go off and change it, or just drop things," and again in the team 3 walkthrough, which "managed to figure 90 percent of it out and that was pretty much it." The remaining 10 percent was manual both times, and it was reliably the part carrying the design detail                                                                                                                                                                                                                                                       |
| Fabricated content inside the artifacts | Tuesday, two teams | Worse than a wrong answer in chat, because it arrives inside the document that is meant to be the specification. One team found contradictory requirements in its own generated requirements document, traced the origin, and got a straight answer: "I went back to Claude and said where did this requirement come from, and it basically told me I made it up." Another found the model had bypassed the supplied mock data and used the real names of retail service providers, despite a project rule against internet calls |
| Accessibility omitted by default        | Tuesday, Team 6    | "We had to prompt Claude to add our accessibility requirements, because initially when the acceptance criteria that it generated, there was no accessibility requirements." Generated acceptance criteria inherit whatever the prompt did not think to ask for, and accessibility is precisely the thing teams do not think to ask for                                                                                                                                                                                            |

## 7.1 The pace problem

One observation deserves its own space because it is a second-order effect nobody was asked about, and it is the kind of thing a methodology can design against. It was raised once on Thursday in general terms and lived out on Tuesday in specifics.

*"If you spend a whole day doing a UI before you hand it over, that is a whole day the developer was sitting still. At the speed a developer can now work, that whole day is a long time. So we are going to have to get really quick at that. If you have got a whole group of people and every one of them feels the need to sprint to keep up, otherwise they are the slowest person in the group, that could actually be a burnout issue. We want to achieve the speed, but we do not want to be causing that."*

The mechanism is specific. When one role gets faster, the handoff intervals that used to be comfortable become the bottleneck, and the pressure lands on whoever has not been accelerated. A lifecycle model that only optimises throughput will produce exactly this. Ours should say something about it.

Tuesday confirms the effect in the other cohort and puts a number on it. One team reported the build itself taking "maybe 20 minutes" against a whole session of process around it, with its engineer idle for most of that: "we probably had people sitting and waiting like mine. I was just waiting quite a while till he could get started."

The same session contains the only working answer anyone produced, and it came from one of the teams. Team 6 stopped treating the lifecycle as a relay. The business analyst, UX designer and product manager worked the acceptance criteria together and generated deliberately rough skeleton screens straight from them, then handed the build over page by page, so the engineer implemented page one while the design pair moved on to page two. "He built those in the background whilst the BA and UX designers kind of moved on to page 2, which kept everything flowing quite nicely."

That is a pipeline rather than a handoff chain, and it is the shape of the answer to this problem. It also cost something, which the same team was honest about: "the review decision making and diligence took longer than expected when Claude kind of owns the creation," and the approach "generated more back and forth than we originally anticipated." The work did not disappear. It moved out of the build and into the review, which is where the next subsection picks it up.

## 7.2 The decision load problem

The most precise critique of the workshop's own framework came from an attendee on Tuesday, and it is aimed squarely at the part of the model we are most likely to adopt without examining it.

The workshop taught the 4D framework from the AI Fluency course: delegation, description, discernment and diligence. Her observation was that the first two get easier and the last two do not, and that the resulting ratio is the problem.

"We felt that there was a lot of information overload. While you were able to defer the first two Ds, the other two Ds, which are decision and diligence, the amount of information that comes out requires you to make so many decisions in a given time frame that you would not usually do on a normal day where you are doing all of the work yourself. Because while you are doing the work, you are also making decisions. I understand the idea is to be able to defer the last two Ds, but I kind of felt like the decision making, because you are making too many decisions while reviewing, there are higher chances of us not being correct with the last two Ds."

She finished with the physical evidence: "I felt tired after the three hour session we did."

This is the pace problem arriving from the other direction. Delegating production does not reduce the work, it converts it into review, and review is a scarcer resource than typing. Alessio agreed without qualification, and his answer was to add more machinery, "that is the part where I potentially go and put in even more technology, an actual governance framework," while conceding that the machinery does not exist: "nobody has that solution yet. I would love to say that I have it, but I am working with AWS, with Microsoft, with IBM, nobody has that solution yet."

Her closing question is the one our model has to take seriously, because it is the one a real adoption programme runs into on week one: "even if companies started adopting it, how is the human going to be able to increase our brain capacity to be able to make these many decisions right on that day, especially with the kind of information as an output that you are getting? And again, in a legacy environment."

## 7.3 What the teams invented

No formal collaboration workflow was taught, so teams improvised their own working arrangements. Four of those approaches are better than anything in the taught material. They came from the audience, not the facilitator, and the client should hear that.

Skeleton first, then pipeline the pages. Described in 7.1. Generate rough screens directly from acceptance criteria before anything is built, then release the build page by page so no role waits on another. It is one promising structural response observed to idle time in either cohort.

The screenshot as the specification. A designer and engineer pair on Tuesday found that listing design defects in prose was slow and unreliable, so they stopped doing it. "What I did on the design page is I just took a screenshot and I dumped the screenshot in, and I basically said to it compare to the screenshot and fix up any elements that are missing." It did not get all the way there, and this is where the 90 percent wall was named, but it replaced a laborious manual diff with a single instruction. For a UX to engineering handoff it is the most directly reusable technique in the recordings.

Accessibility encoded as a skill. Team 6 ran an external accessibility checker against their first page, found five or six WCAG 2.2 AAA failures, fixed them with the model, and then wrote the standard into a skill so that later pages were generated against it. The issue count on the next page "drastically reduced." This is the only case in either cohort of a team turning a review finding into a control that applies automatically next time, and it is a small worked example of the yellow to green migration described in section 4.

Acceptance criteria as a shared artifact in version control. Several Tuesday teams arrived at the same pattern independently. The business analyst writes the acceptance criteria, commits them into the repository as Markdown, and everyone else references that file by name in their prompts instead of pasting context around. It routes around the collaboration gap by using Git as the shared memory the tool does not provide, and one Thursday team reached the same place from the other direction.

None of these were taught. All four are candidates for our model, and the fact that they were invented under time pressure by people using the tools for the second day of their lives is an argument for their practicality rather than against it.

## 7.4 The design role specifically

Group 1 leaned on Claude Design more heavily, so Tuesday contains the most detailed examination of the design workflow in the recordings, including discussion from participants actually doing the design work. That matters disproportionately for Team 2, because stage 4 of the eight is ours.

The designers who spoke described the same experience. The tool is fast at producing something to react to, and poor at the part designers consider the real work. "It is interesting prompting from the beginning and getting into the prototype, that it is very fiddly and it takes a lot of time to get very fundamental design things on the page. That you could do quite a lot more efficiently using something like a canvas and being able to move design elements, instead of writing to a micro level, because you have to really understand what each component needs to be, or headings versus microcopy versus subcopy. So it is very literal in its sense and very structured in that way."

That is a precise claim and it is testable: prose is a poor input device for spatial decisions. It is also why the screenshot technique in 7.3 works, because a screenshot is spatial and a paragraph is not.

The second finding is degradation across a document rather than across a session. "If you are 20 rows deep it sort of loses its place, or if you are three pages in it deletes something from the first page and you are like, no, what are you doing." Section 4.5 explains the mechanism. It bites design work earlier than it bites code, because a design system is exactly the kind of quiet background consistency that lives in the part of the context that gets compacted away first.

The third is homogenisation, and the designer and the facilitator arrived at it independently. "If you want vanilla, an AI agent is more than able to create vanilla. But if you want something that stands out, is more robust, that is when you need designers with a lot of experience to push that, to elevate it." Alessio's version was blunter: "the first thing about telling you if a website is made by AI, if it is purple. If it is purple it is made by AI, because they were all trained on purple websites." The point underneath is that unconstrained generation converges, so in an AI lifecycle the design system stops being decoration and becomes the mechanism that keeps output from being generic. That is an argument for loading design tokens into the context deliberately and early, which is what Alessio ended up doing by hand when the brand colours went wrong in 4.3.

One practical note for our own tooling choices. Claude Design left beta roughly three weeks before the workshop, it now carries connectors and can export to Figma, and the facilitator's read was that the

position is unsettled: "whether or not Figma will still exist in one year, that will have to be seen." Treat that as a reason to keep our design artifacts portable, not as a prediction.

None of this displaces the designer, and the framing that survived both days is the one from the workshop's own material: the designer moves from producer to director. What the recordings add is that directing is harder than producing when the medium of direction is prose, and that the roles now reviewing generated output all day are the ones most exposed to the decision load problem in 7.2.

## 7.5 Two questions left open

Two questions from the floor received partial answers but remained unresolved in practice.

The first, from an attendee named Ben: is collaboration less necessary under an agentic model, or was it only hard because this was a simulation? The answer given was that teams need one shared tool, notifications on who changed what, and clean handovers, after which "you can do many steps in parallel." That is an assertion, not a demonstration.

The second, from Karen, is more radical: does documentation stop being the artifact? "You could picture having a discussion with the business and walking away with some requirements, and you quickly build a prototype and you say here it is, and it might be 80 percent right. You get the comments on that, and then the comments become the spec." She contrasted three hours of work against what would normally take the better part of a program increment.

The counter came immediately and it is the right one: this works for a screen and not for a backend change, and there are systems nobody should point it at.

## 8. The audience problem

Team 2 has been treating the audience split as an open decision with no evidence behind it. There is evidence, and it points somewhere more useful than either option.

*Alessio described the problem in his closing remarks without being asked: "Monday and Tuesday we tried to make things very technical, and then we ended up with people complaining that it was too technical, because we had most of the non technology people there. So we changed everything for Wednesday and Thursday and said, let us make it more non technical friendly. So yes, non tech friendly, and we hit the opposite."*

Two cohorts, two mismatches, opposite directions. The workshop was not too advanced or too basic. It was single-layered.

Tuesday shows that decision being taken rather than described afterwards, which makes it stronger evidence. At the end of the Group 1 build day, before Wednesday had started, Alessio told the room the next cohort would get something different: "for tomorrow we are going to redesign the workshop a little bit based on your feedback, because we are finding that perhaps the actual talking about the technology as a development technology is not as interesting or as fun as if we talk about everything else apart from the development." The correction was made in good faith, and it overshot. That is a single-layer problem, not a facilitation failure, and no amount of skill at reading a room fixes it.

The two audiences also fail differently, and the transcripts are specific about how.

|                             | Junior and non-technical                                                                                                    | Experienced engineers                                                                                                                        |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Where they get stuck        | Terminal use, whether VS Code or GitHub is needed, sharing files, getting a machine configured, knowing what the words mean | Complex conditional logic, enterprise payloads, enhancing an API that has existed for years, reviewing generated code that reads as unusable |
| What good looks like        | Getting to a working artifact at all                                                                                        | Getting to code that survives review                                                                                                         |
| What the workshop gave them | A demonstration that assumed the setup was done                                                                             | A demonstration of a problem they can solve in half an hour by hand                                                                          |
| Their own words             | "There is a lot of technical knowledge required just using the tools, especially if you are coming in cold"                 | "It did the boilerplate very, very well, but the code itself was absolute garbage"                                                           |

A model built for one of these does not serve the other, which is what was observed for both cohorts in the workshop.

The recommendation for Sprint 1 is to keep the junior angle as Team 2's primary audience, because the evidence for it is more concrete and there is more of it, and Team 1 can take the experienced end. But the two models need a shared spine. The red, yellow and green classification and the eight stages work for both. What differs is what sits inside each box: setup, vocabulary and collaboration mechanics for one, verification, legacy and scale for the other.

That framing is also more defensible to the client than picking a side, because it is what his own workshop data says.

## 9. NBN ways of working, as far as we can see them

The brief says NBN already has ways of working and our model has to fit alongside them. The recordings give a partial picture, gathered incidentally.

The delivery model is role-based and recognisably waterfall in its handoffs. One attendee said so directly while describing the exercise: "it is still quite waterfall this process, handing over documentation to different members of the team." Teams were assigned PM, BA, UX, developer and tester roles because that is how NBN works, and several teams abandoned the roles under time pressure, which is its own finding.

Requirements live in Jira or Confluence. That came up as a question about integration and was not resolved.

Microsoft is the default environment. Teams for collaboration, Copilot as the AI tool people had already used at work. Several attendees had used Copilot and nothing else.

Git is not universal. "Not everybody has GitHub or needs GitHub. As long as the developer has it on their machine." That is a significant constraint for any model assuming a Git-centred lifecycle, and it is one of the several repeatedly reported constraint that broke collaboration for most teams.

Ordinary corporate boundaries broke the tooling in ordinary ways, and both instances cost build time. One Tuesday team could not exchange Word documents at all because the participants sat in different

tenants, and worked around it by having the model convert everything to Markdown and pasting that instead. Another was blocked pushing to the shared repository because the engineer was authenticated with a personal account. Neither is an AI problem. Both are the kind of friction a lifecycle model has to assume rather than wish away.

NBN already has accessibility tooling and expects it to be used. A team reached for Siteimprove as a browser extension without being prompted, called it a "very, very efficient tool in checking accessibility," and treated a WCAG 2.2 AAA pass as part of their definition of done. Our model should assume that standard already exists rather than introduce it.

There are systems that are off limits. The closing discussion named several classes of system that nobody would point an AI at, and there is a guardrail against putting the NBN name into prompts.

The environment is large and old. Alessio's framing of decades of legacy written by different people in different languages went unchallenged in a room of NBN engineers.

## 10. Reusable against needs rebuilding

The question this report exists to answer is how much of the workshop we can reuse and how much needs replacing.

### 10.1 Take this forward

- The red, yellow and green classification. It is already delivered to the client, they have seen it, and it is a better spine than anything we would invent. Use it.
- The claim that the boundaries migrate as governance and context improve. This turns a static classification into something a team can act on, and it gives the governance module a purpose.
- The eight-stage breakdown. Conventional, which is the point, because it maps onto how NBN already describes delivery.
- CLAUDE.md as the place organisational rules accumulate, and the pattern of correcting the context rather than the output.
- Issue discipline: acceptance criteria as the contract, explicit out-of-scope as the defence against scope creep.
- The principle that context quality determines output quality, and specifically that an infrastructure map is what moves a stage from red to yellow.
- The fault reporting tool as a worked example. It is already familiar to everyone who attended, which makes it a strong candidate for our own demonstration.
- Hooks as the enforcement mechanism, and the principle underneath them: an instruction in a prompt or a rules file is probabilistic, so anything that must not happen has to be enforced outside the model.
- Skills as the place a review finding becomes a standing control. The accessibility skill one team built is the smallest complete example we have of a task migrating from yellow to green.
- The screenshot as the design specification, for the UX to engineering handoff.
- Skeleton first and pipelined page delivery, as the answer to role idle time.
- Sub-agents as the mitigation for context degradation, on the reasoning that a fresh sub-agent carries no poisoned context.

## 10.2 Build this ourselves

- Collaboration. Nothing was taught, and the facilitator agreed nothing adequate exists: the tool is "not something that seems ready for proper group collaboration." But the teams invented four workable patterns between them, so the job is to formalise and test what the audience already found rather than to start from a blank page. Still the biggest single gap and still the one most likely to make our model valuable.
- Liability and authorship. Technical guardrails are now partly answered in section 4.5, and the answer is that the model cannot police itself. What is still entirely missing is the human half: who signs off machine-written code, who is accountable when it fails, and what authorship means for it. The brief calls these first-class and neither cohort raised them.
- Verification. "Are all the tests correct?" is still the right question and still unanswered. Tuesday adds one data point in favour, a generated test suite that caught the model substituting real provider names for the supplied mock data, and one against, a requirements document that contradicted itself because the model had invented a requirement. Both argue for verification as a taught discipline rather than a hope.
- Legacy and enhancement work. Named as the real first use case for a large organisation, then skipped. For an audience whose day job is enhancing an API that has existed for years, this is the whole job.
- Deployment. Stage 8 of 8, never reached.
- Depth for senior engineers. The complex-logic challenge from the floor was conceded and never answered.
- Layering. The single-audience pitch is what broke the workshop twice.
- Decision load. The workshop's own framework was challenged on the grounds that it converts production work into review work faster than people can absorb, and the facilitator said that he was not aware of any existing solution. A lifecycle model that measures only throughput will optimise straight into this bottleneck.

## 11. What this changes for Sprint 1

What this report changes for the sprint, listed so the board can be updated.

| Change                                 | Action                                                                                                                                                                                                                                   |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The audience question has evidence now | Take it to the client as a recommendation with the workshop's own data behind it, rather than as an open question. Team 2 on juniors, Team 1 on experienced, shared spine                                                                |
| D3 has a starting point                | The vendor research in PRD-3 should be compared against the red, yellow and green model rather than gathered in a vacuum. The question becomes what Google, Atlassian, Microsoft and Anthropic add to a framework the client already has |
| The lifecycle map has its boxes        | The eight stages are a defensible starting sequence for D4, and each box needs a classification and a note on what would move it                                                                                                         |



| Change                                                        | Action                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Collaboration becomes a module                                | It was not in the module list because nobody had read the transcripts. On this evidence it is not optional                                                                                                                                                                                                                                  |
| The demonstration project has a candidate                     | The fault reporting tool is already understood by the client and by everyone who attended                                                                                                                                                                                                                                                   |
| Two questions for Alessio                                     | Narrowed now that Tuesday is recovered. Are these four recordings the full set, given there is no session 1 for Monday or Tuesday and nothing at all for Wednesday. And is the post-workshop survey available: he told the Tuesday room "expect a survey", so it should exist, and it would be direct evidence of what the audience thought |
| Section 7 now rests on twelve teams, not six                  | Every obstacle can be reported as replicated across both cohorts or specific to one. That is a materially stronger claim to put in front of the client and it should be stated in the summary, not buried                                                                                                                                   |
| Collaboration starts from candidate answers, not a blank page | It remains a module, but Sprint 2 should test the four patterns the teams invented rather than design from scratch. Skeleton-first pipelining and the screenshot-as-spec handoff are the two worth prototyping first                                                                                                                        |
| Governance has a technical spine                              | Section 4.5 gives D4 a defensible position: rules files are probabilistic, hooks and external checks are not, and no agent should be asked to police itself. That is a sharper claim than anything in the vendor material and it came from the client's own facilitator                                                                     |
| Decision load belongs in the model                            | The strongest critique of the framework came from the floor and was conceded. Our lifecycle map needs a review-capacity dimension, not just a classification per stage                                                                                                                                                                      |
| A standing invitation is open                                 | Alessio offered the Tuesday room a follow-up session at RMIT and a round table later in the year on enterprise-scale and legacy code. Worth taking up while the capstone is live, since it is the exact gap D2 identifies                                                                                                                   |

*Sources: NBN workshop recordings, June 2026, shared by Alessio Bonti. Monday afternoon, Tuesday session 2, Thursday sessions 1 and 2. Transcribed via Microsoft Stream and cleaned; the Tuesday transcript was regenerated in English on 13 August 2026 after the original was produced through a French model, and it supersedes the version described in the coverage note. Transcripts and coverage notes are in the Workshop Transcripts folder. Supporting context from the client meeting of 10 August 2026 and the supervisor session of 12 August 2026.*